Grad Cafe Query Analysis

JHU Software Concepts in Python - Module 3

Q1: How many entries do you have in your database who have applied for Fall 2026?

Answer:

Applicant count: 33,052

SQL Query:

```
SELECT COUNT(*)
FROM applicants
WHERE term = 'Fall 2026';
```

Description:

I used COUNT(*) with a WHERE clause filtering on term = 'Fall 2026' to count every row in the applicants table that belongs to that term.

Q2: What percentage of entries are from International students (not American or Other) (to two decimal places)?

Answer:

Percent International: 46.23%

SQL Query:

```
SELECT ROUND(
    100.0
    * COUNT(CASE WHEN us_or_international = 'International' THEN 1 END)
    / NULLIF(
        COUNT(CASE WHEN us_or_international IN ('International', 'American') THEN 1 END),
        0
    ),
    2
) AS pct_international
FROM applicants;
```

Description:

I counted rows where us_or_international = 'International' using CASE WHEN inside COUNT, then divided by the count of rows where us_or_international is either 'International' or 'American'. This excludes NULL entries and any 'Other' values from both numerator and denominator, so the percentage reflects only the known international vs. American entries. ROUND(..., 2) gives two decimal places, and NULLIF prevents a divide-by-zero error.

Q3: What is the average GPA, GRE, GRE V, GRE AW of applicants who provided these metrics?

Answer:

Average GPA: 3.78 | Average GRE: 262.00 | Average GRE V: 161.54 | Average GRE AW: 9.30

SQL Query:

```
SELECT
    ROUND(AVG(gpa)::numeric, 2)    AS avg_gpa,
    ROUND(AVG(gre)::numeric, 2)    AS avg_gre,
    ROUND(AVG(gre_v)::numeric, 2)  AS avg_gre_v,
    ROUND(AVG(gre_aw)::numeric, 2) AS avg_gre_aw
FROM applicants;
```

Description:

I used AVG() on each score column. PostgreSQL's AVG() automatically ignores NULL values, so each average is computed only over entries that actually reported that metric. The ::numeric cast is required because PostgreSQL's ROUND function does not accept FLOAT arguments directly.

Q4: What is the average GPA of American students in Fall 2026?

Answer:

Average GPA American: 3.79

SQL Query:

```
SELECT ROUND(AVG(gpa)::numeric, 2) AS avg_gpa
FROM applicants
WHERE us_or_international = 'American'
  AND term = 'Fall 2026'
  AND gpa IS NOT NULL;
```

Description:

I filtered the AVG query to only include rows where the nationality is 'American', the term is 'Fall 2026', and the gpa column is not NULL.

Q5: What percent of entries for Fall 2026 are Acceptances (to two decimal places)?

Answer:

Acceptance percent: 35.84%

SQL Query:

```
SELECT ROUND(
    100.0
    * COUNT(CASE WHEN status = 'Accepted' THEN 1 END)
    / NULLIF(COUNT(*), 0),
    2
) AS pct_accepted
FROM applicants
WHERE term = 'Fall 2026';
```

Description:

I filtered to Fall 2026 entries with WHERE, then used CASE WHEN inside COUNT to count only accepted rows, dividing by the total count of Fall 2026 entries. NULLIF(COUNT(*), 0) guards against dividing by zero.

Grad Cafe Query Analysis

JHU Software Concepts in Python - Module 3

Q6: What is the average GPA of applicants who applied for Fall 2026 who are Acceptances?

Answer:

Average GPA Acceptance: 3.78

SQL Query:

```
SELECT ROUND(AVG(gpa)::numeric, 2) AS avg_gpa
FROM applicants
WHERE term = 'Fall 2026'
  AND status = 'Accepted'
  AND gpa IS NOT NULL;
```

Description:

I averaged the gpa column filtering only for Fall 2026 entries with an Accepted status and a non-NULL gpa value.

Q7: How many entries are from applicants who applied to JHU for a Masters degree in Computer Science?

Answer:

JHU Masters CS count: 13

SQL Query:

```
SELECT COUNT(*) FROM applicants
WHERE (
    program ILIKE '%Johns Hopkins%'
    OR program ILIKE '%JHU%'
)
  AND program ILIKE '%Computer Science%'
  AND degree = 'Masters';
```

Description:

I used ILIKE with % wildcards to search the program column for 'Johns Hopkins' or 'JHU', and also for 'Computer Science' in the same field. ILIKE performs a case-insensitive match, which helps catch entries with inconsistent capitalization. I also filtered degree = 'Masters' to exclude PhD entries.

Q8: How many entries from 2026 are acceptances from applicants who applied to Georgetown University, MIT, Stanford University, or Carnegie Mellon University for a PhD in Computer Science? (raw downloaded fields)

Answer:

Count (raw fields): 30

SQL Query:

```
SELECT COUNT(*) FROM applicants
WHERE term ILIKE '%2026%'
  AND status = 'Accepted'
  AND degree = 'PhD'
  AND program ILIKE '%Computer Science%'
  AND (
      program ILIKE '%Georgetown%'
      OR program ILIKE '%Massachusetts Institute of Technology%'
      OR program ILIKE '%MIT%'
      OR program ILIKE '%Stanford%'
      OR program ILIKE '%Carnegie Mellon%'
  );
```

Description:

I searched the raw program column using ILIKE for each school name and for 'Computer Science', combined with filters for Accepted status, PhD degree, and a term containing '2026'. I included both 'MIT' and 'Massachusetts Institute of Technology' as search terms to account for different ways entries might spell the school name.

Q9: Do your numbers for question 8 change if you use LLM Generated Fields?

Answer:

Count (LLM fields): 30 -- No change from Q8.

SQL Query:

```
SELECT COUNT(*) FROM applicants
WHERE term ILIKE '%2026%'
  AND status = 'Accepted'
  AND degree = 'PhD'
  AND llm_generated_program ILIKE '%Computer Science%'
  AND (
      llm_generated_university ILIKE '%Georgetown%'
      OR llm_generated_university ILIKE '%Massachusetts Institute of Technology%'
      OR llm_generated_university ILIKE '%MIT%'
      OR llm_generated_university ILIKE '%Stanford%'
      OR llm_generated_university ILIKE '%Carnegie Mellon%'
  );
```

Description:

I ran the same query as Q8 but replaced the program column with llm_generated_program and searched llm_generated_university for the school names. The result is the same (30), which suggests the LLM standardization did not introduce or remove entries for these schools compared to the raw data.

Q10: What are the top 10 most applied-to universities in the dataset?

Answer:

See ranked list below.

1. Stanford University: 796 entries
2. University of California, Berkeley: 753 entries
3. University of Washington: 660 entries
4. Yale University: 645 entries
5. University of Wisconsin-Madison: 621 entries
6. University of Michigan, Ann Arbor: 616 entries
7. University of Toronto: 612 entries
8. Princeton University: 608 entries
9. University of Chicago: 608 entries
10. Massachusetts Institute of Technology: 590 entries

SQL Query:

```
SELECT llm_generated_university, COUNT(*) AS entry_count
FROM applicants
WHERE llm_generated_university IS NOT NULL
GROUP BY llm_generated_university
ORDER BY entry_count DESC
LIMIT 10;
```

Description:

I grouped all entries by llm_generated_university, counted the entries per university with COUNT(*), and sorted in descending order to get the top 10. I used the LLM-generated field because it produces more consistent university names across entries, making GROUP BY more reliable.

Q11: Which 10 universities have the lowest acceptance rate (among universities with at least 15 entries)?

Answer:

See ranked list below.

1. Loyola University Maryland: 4.17% (24 total entries)
2. Northwestern University Feinberg School of Medicine: 5.56% (18 total entries)
3. Suffolk University: 5.88% (17 total entries)
4. Weill Cornell Medicine: 9.68% (31 total entries)
5. Emory University: 11.73% (179 total entries)
6. University of California, San Francisco: 13.7% (73 total entries)
7. University of Wisconsin: 15.0% (40 total entries)
8. Yale University: 15.04% (645 total entries)
9. Rowan University: 16.67% (18 total entries)
10. University of North Carolina: 17.44% (86 total entries)

SQL Query:

Grad Cafe Query Analysis

JHU Software Concepts in Python - Module 3

```
SELECT
    llm_generated_university,
    ROUND(
        100.0 * COUNT(CASE WHEN status = 'Accepted' THEN 1 END) / COUNT(*),
        2
    ) AS acceptance_rate,
    COUNT(*) AS total_entries
FROM applicants
WHERE llm_generated_university IS NOT NULL
GROUP BY llm_generated_university
HAVING COUNT(*) >= 15
ORDER BY acceptance_rate ASC
LIMIT 10;
```

Description:

I calculated the acceptance rate per university as accepted entries divided by total entries, grouped by llm_generated_university. HAVING COUNT(*) >= 15 filters out universities with very few entries so the rates are based on enough data to be meaningful. The results are sorted ascending to show the lowest acceptance rates first.